JEDIEL ELIEZER SANTOS DE OLIVEIRA
LUCAS ABATI ZANOTTO

FOGESCRIPT

Uma linguagem de programação inspirada no meme
“A senhora trabalha na Assembleia?”

Documento de especificação apresentado à disciplina de Compiladores, como parte da atividade “Do meme ao código”, com a definição da linguagem FogeScript e do seu tradutor para JavaScript.

2026

SUMÁRIO

1 INTRODUÇÃO **3**

2 O MEME E A ESCOLHA DO TEMA **3**

2.1 Origem do meme 3

2.2 Justificativa 3

2.3 Diferenças em relação à BIRL 4

3 A LINGUAGEM FOGESCRIPT **4**

3.1 Tabela de comandos 4

3.2 Estrutura de um programa 5

3.3 Operadores 5

3.4 Tipos de dados 5

3.5 Exemplo de programa 6

4 ANÁLISE LÉXICA **6**

4.1 Definição dos tokens 7

4.2 Regras de reconhecimento 7

4.3 Exemplo 7

4.4 Erros léxicos 8

5 ANÁLISE SINTÁTICA **8**

5.1 Gramática formal 9

5.2 Árvore sintática 10

5.3 Erros sintáticos 10

6 ANÁLISE SEMÂNTICA **11**

6.1 Escopos e tabela de símbolos 11

6.2 Inferência de tipos 12

6.3 Verificações 13

7 GERAÇÃO DE CÓDIGO **14**

8 TESTES E EVIDÊNCIAS DE FUNCIONAMENTO **15**

8.1 Programas de teste 15

8.2 Execução dos testes 16

8.3 Evidência 1: entrada de dados, decisões e repetição 16

8.4 Evidência 2: funções, listas e laço para 18

8.5 Evidência 3: erro na hora de rodar 21

9 COMO COMPILAR E EXECUTAR **21**

10 CONSIDERAÇÕES FINAIS **22**

REFERÊNCIAS **23**

1 INTRODUÇÃO

Este documento apresenta a FogeScript, a linguagem de programação que criamos para a atividade “Do meme ao código” da disciplina de Compiladores. A linguagem foi inspirada no meme “A senhora trabalha na Assembleia?”, e todas as palavras reservadas são falas ou situações do vídeo.

Junto com a linguagem desenvolvemos um tradutor, que recebe um programa escrito em FogeScript (um arquivo com extensão `.foge`), faz a análise léxica, a análise sintática e a análise semântica e, se não encontrar erros, gera um programa equivalente em JavaScript. O programa gerado pode ser executado com o Node.js.

Escolhemos o JavaScript como linguagem de destino porque assim o mesmo tradutor funciona em dois lugares: na linha de comando, com o Node.js, e no navegador. No site do projeto (https://fogescript.vercel.app) dá para escrever um programa, ver o resultado de cada etapa da tradução e rodar o código gerado. O tradutor também foi escrito em JavaScript e não usa bibliotecas externas, ou seja, o analisador léxico, o analisador sintático e a análise semântica foram programados pelo grupo. O código-fonte está no GitHub, em https://github.com/improvez/FogeScript.

A seção 2 explica o meme e por que ele foi escolhido. A seção 3 descreve os comandos, operadores e tipos da linguagem, e as seções 4 a 7 explicam cada etapa do tradutor. Na seção 8 estão os programas de teste e as evidências de funcionamento, e na seção 9 as instruções para compilar e executar o projeto.

2 O MEME E A ESCOLHA DO TEMA

2.1 Origem do meme

O meme vem de um vídeo de reportagem que circulou muito nas redes sociais. Na gravação, uma mulher é abordada na rua e perguntam a ela se trabalha na Assembleia. Ela nega, diz que nunca fez aquilo e sai correndo, enquanto a equipe de reportagem vai atrás chamando por ela. A pergunta “A senhora trabalha na Assembleia?” e outras falas do vídeo viraram bordão na internet brasileira.

2.2 Justificativa

Escolhemos esse meme porque ele é bem conhecido e as falas são curtas, então quem já viu o vídeo reconhece as frases quando lê um programa. Além disso, a história do vídeo tem uma sequência que combina com a estrutura de um programa. A pergunta da reportagem abre o programa (`A SENHORA TRABALHA NA ASSEMBLEIA?`) e a fuga fecha cada bloco (`FUGIU`). A perseguição virou os laços de repetição (`ENQUANTO TIVER CORRENDO` e `SENHORA? SENHORA?!`), e o “volta aqui!” é o comando que interrompe o laço. A negação “não, eu nunca fiz isso” declara uma função, e o que foi gravado pela câmera é a saída do programa (`TEMOS GRAVADO!`).

O meme também serviu para o tratamento de erros, que não era obrigatório: o repórter faz a pergunta (`DEIXA EU TE PERGUNTAR`, que equivale ao try), a senhora fica nervosa (`A SENHORA TÁ NERVOSA?`, o catch) e pede para ser deixada em paz (`ME DEIXA EM PAZ!`, o throw).

2.3 Diferenças em relação à BIRL

A BIRL foi a inspiração do trabalho, mas tentamos não copiar a sintaxe dela. Na FogeScript as palavras reservadas são frases inteiras, com espaço e pontuação, e o analisador léxico reconhece cada frase como um único token, com ou sem acento e em maiúsculas ou minúsculas. Existe um único terminador de bloco (`FUGIU`), usado também no se/senão, e a declaração de variável não tem tipo, porque quem descobre o tipo é a análise semântica. Também incluímos listas, try/catch e algumas funções prontas (`TAMANHO`, `NUMERO`, `INTEIRO`, `TEXTO` e `ALEATORIO`).

3 A LINGUAGEM FOGESCRIPT

3.1 Tabela de comandos

Tabela 1 – Comandos da linguagem FogeScript

Expressão do meme	O que faz	Tradução para JavaScript
`A SENHORA TRABALHA NA ASSEMBLEIA?`	Início do programa	`function assembleia() {`
`FUGIU`	Fim de bloco	`}`
`A SENHORA FEZ x = 1;`	Declara uma variável	`let x = 1;`
`TEMOS GRAVADO!(a, b);`	Saída (print)	`foge.gravar(a, b);`
`QUER CONVERSAR COMIGO?(x);`	Entrada (input)	`x = foge.ler(linha);`
`SE A SENHORA NÃO É FUNCIONÁRIA (c)`	Se (if)	`if (c) {`
`ENTÃO A SENHORA É O QUÊ? (c)`	Senão se (else if)	`} else if (c) {`
`E TÁ CORRENDO POR QUÊ?`	Senão (else)	`} else {`
`ENQUANTO TIVER CORRENDO (c)`	Enquanto (while)	`while (c) {`
`SENHORA? SENHORA?! (ini; c; passo)`	Para (for)	`for (ini; c; passo) {`
`VOLTA AQUI!`	Sai do laço (break)	`break;`
`DEIXA ELA CORRER`	Pular (continue)	`continue;`
`NÃO, EU NUNCA FIZ ISSO f(a, b)`	Declara uma função	`function f(a, b) {`
`CHAMA A SENHORA f(1, 2)`	Chamada de função	`foge.chamar("f", [1, 2], linha)`
`CHEGA DE CORRER valor;`	Retorno (return)	`return valor;`
`TÁ GRAVADO`	Verdadeiro	`true`
`NÃO TÁ GRAVADO`	Falso	`false`
`DEIXA EU TE PERGUNTAR`	Tentar (try)	`try {`
`A SENHORA TÁ NERVOSA? (e)`	Captura o erro (catch)	`} catch (e) {`
`ME DEIXA EM PAZ! "msg";`	Lança um erro (throw)	`throw foge.lancar("msg", linha);`

A Tabela 1 mostra a correspondência entre as expressões do meme e os comandos de programação, e como cada comando é traduzido para JavaScript. O `FUGIU` fecha qualquer bloco: o programa, o se, os laços, as funções e o try. Na chamada de função, o `CHAMA A SENHORA` é opcional, então `f(1, 2)` também funciona. Na tradução, `linha` é o número da linha no programa FogeScript, usado nas mensagens de erro, e as funções que começam com `foge.` fazem parte do runtime, explicado na seção 7.

3.2 Estrutura de um programa

Todo programa tem um bloco principal, que começa com `A SENHORA TRABALHA NA ASSEMBLEIA?` e termina com `FUGIU`. Antes ou depois desse bloco podem vir declarações de funções e de variáveis globais. O ponto e vírgula no fim dos comandos é opcional. As palavras reservadas podem ser escritas em maiúsculas ou minúsculas, com ou sem acento, mas os nomes de variáveis diferenciam maiúsculas de minúsculas.

Os comentários seguem o formato do C e do JavaScript: `//` comenta até o fim da linha e `/* ... */` comenta um trecho de várias linhas.

3.3 Operadores

Tabela 2 – Operadores

Tipo	Operadores	Exemplo
Aritméticos	`+ - * / %` (resto)	`total / dias`
Relacionais	`== != < > <= >=`	`idade >= 18`
Lógicos	`&&` (e), `||` (ou), `!` (não)	`!acertou && tentativas < 5`
Atribuição	`= += -= *= /= %=`	`x += 5`
Incremento e decremento	`++ --`	`i++`

O `+` também junta textos, e um texto somado com outro valor vira texto (`"Dia " + 3` dá `"Dia 3"`).

3.4 Tipos de dados

Tabela 3 – Tipos de dados

Tipo	Exemplos	Observações
número	`37`, `8800.50`, `-1`	Inteiros e números com casas decimais
texto	`"Dona Cida"`, `'oi'`	Aspas duplas ou simples. Aceita `\n`, `\t` e `\"`. `T[0]` lê a primeira letra
booleano	`TÁ GRAVADO`, `NÃO TÁ GRAVADO`	Verdadeiro e falso
lista	`[1, 2, 3]`, `[]`	`L[i]` lê ou altera uma posição. Pode misturar tipos
nada	(sem literal)	Valor de uma variável declarada sem valor

A tipagem é dinâmica: a declaração não diz o tipo e uma variável pode receber valores de tipos diferentes. Mesmo assim, a análise semântica descobre o tipo de cada variável pelos valores que ela recebe e já acusa, antes de rodar, as operações em que os tipos não combinam (seção 6). O comando de entrada transforma a resposta em número quando ela parece um número, então `"37"` vira `37` e `"2,5"` vira `2.5`.

A linguagem tem algumas funções prontas para conversão: `NUMERO(x)` e `INTEIRO(x)` transformam texto em número, `TEXTO(x)` transforma qualquer valor em texto, `TAMANHO(x)` devolve o número de letras de um texto ou de itens de uma lista e `ALEATORIO(a, b)` sorteia um inteiro entre `a` e `b`.

3.5 Exemplo de programa

O programa abaixo é o `somador.foge`, da pasta `exemplos`. Ele declara uma função, lê dois números, mostra a soma e usa um se/senão.

Código 1 – Programa exemplos/somador.foge

```
// O somador da BIRL, agora na Assembleia
NÃO, EU NUNCA FIZ ISSO SOMAR(A, B)
    CHEGA DE CORRER A + B;
FUGIU

A SENHORA TRABALHA NA ASSEMBLEIA?
    A SENHORA FEZ A, B, RES;
    TEMOS GRAVADO!("Me passa dois números aí, senhora:");
    QUER CONVERSAR COMIGO?(A, B);
    RES = CHAMA A SENHORA SOMAR(A, B);
    TEMOS GRAVADO!("O resultado é", RES);

    SE A SENHORA NÃO É FUNCIONÁRIA (RES == 37)
        TEMOS GRAVADO!("37! Tá gravado!");
    E TÁ CORRENDO POR QUÊ?
        TEMOS GRAVADO!("Volta aqui, senhora!");
    FUGIU
FUGIU
```

Rodando o programa traduzido com a entrada 30 e 7:

Código 2 – Execução do somador.foge traduzido

```
$ node somador.js < somador.entrada
Me passa dois números aí, senhora:
O resultado é 37
37! Tá gravado!
```

4 ANÁLISE LÉXICA

O analisador léxico é a função `tokenize`, no arquivo `fogescript.js`. Ele lê o código caractere por caractere, da esquerda para a direita, e agrupa os caracteres em tokens. Cada token guarda a categoria, o texto lido, a linha e a coluna em que começa. Espaços, quebras de linha e comentários só servem para separar os tokens e são descartados.

4.1 Definição dos tokens

Tabela 4 – Categorias de tokens

Categoria	Padrão (expressão regular)	Exemplos
Palavra reservada	uma das 20 frases da Tabela 1	`FUGIU`, `A SENHORA FEZ`
Identificador	`[\p{L}_][\p{L}\p{N}_]*`	`salario`, `dias_uteis`, `ação`
Número	`[0-9]+(\.[0-9]+)?`	`37`, `8800.50`
Texto	`"([^"\\\n]|\\.)*"` ou o mesmo com `'`	`"Bom dia!"`
Operador	os operadores da Tabela 2	`<=`, `+=`
Delimitador	`( ) [ ] , ;`	`(`, `;`
Comentário (descartado)	`//[^\n]*` e `/\*(.|\n)*?\*/`	`// fim de semana`

4.2 Regras de reconhecimento

Como as palavras reservadas são frases com espaços, o analisador não pode simplesmente ler uma palavra de cada vez. Em cada posição ele testa todas as frases reservadas e fica com a mais longa que casar (regra do maior casamento). Só quando nenhuma frase casa é que ele lê um identificador. Por isso `NÃO TÁ GRAVADO` vira um único token, o valor falso, e não o identificador `NÃO` seguido do verdadeiro `TÁ GRAVADO`.

Na comparação com as frases, maiúsculas, minúsculas e acentos não fazem diferença (`nao ta gravado` é o mesmo que `NÃO TÁ GRAVADO`), e as palavras da frase podem estar separadas por mais de um espaço ou por uma quebra de linha. Uma frase que termina em letra só é aceita se o próximo caractere não continuar o nome, então `FUGIU_MESMO` é um identificador comum.

4.3 Exemplo

Para o trecho do Código 3, o analisador produz os tokens da Tabela 5. O comentário não aparece na lista porque é descartado.

Código 3 – Trecho usado no exemplo da análise léxica

```
A SENHORA FEZ salario = 8800.50;  // este comentário é descartado
SE A SENHORA NÃO É FUNCIONÁRIA (salario >= 1000)
```

Tabela 5 – Tokens produzidos pelo analisador léxico para o trecho

Linha:coluna	Categoria	Texto lido	Valor
1:1	palavra reservada	`A SENHORA FEZ`	`VAR`
1:15	identificador	`salario`	`salario`
1:23	operador	`=`	`=`
1:25	número	`8800.50`	`8800.5`
1:32	delimitador	`;`	`;`
2:1	palavra reservada	`SE A SENHORA NÃO É FUNCIONÁRIA`	`IF`
2:32	delimitador	`(`	`(`
2:33	identificador	`salario`	`salario`
2:41	operador	`>=`	`>=`
2:44	número	`1000`	`1000`
2:48	delimitador	`)`	`)`

4.4 Erros léxicos

O analisador léxico informa erro quando encontra um símbolo que não faz parte da linguagem (como `@`, `#` ou `{`), um texto sem as aspas de fechamento, um comentário `/*` sem o `*/` ou um número grudado em letras, como `2dias`, que não é número nem nome válido. A mensagem diz a linha e a coluna do erro e mostra a linha do código com uma marca embaixo da posição. O Código 4 mostra a mensagem do tradutor para um dos programas de teste (na seção 9 aparece a saída completa do tradutor, com o resultado de cada etapa).

Código 4 – Erro léxico: símbolo que não pertence à linguagem

```
Erro léxico na linha 6, coluna 30: Símbolo estranho "@": ele não faz parte da linguagem
  6 |     A SENHORA FEZ total = 10 @ 2;
    |                              ^
```

Os outros dois testes de erro léxico, com texto sem fechar e nome começando com número, estão na pasta `testes/erros` e aparecem na Tabela 9.

5 ANÁLISE SINTÁTICA

O analisador sintático é a função `parse`. Ele recebe a lista de tokens e confere se ela segue a gramática da linguagem. Fizemos um analisador descendente recursivo: cada regra da gramática virou uma função, que consome os tokens da regra e chama as funções das regras de dentro. O resultado é uma árvore sintática abstrata (AST), que é usada nas etapas seguintes.

5.1 Gramática formal

A gramática está em EBNF. Na notação usada, `{ x }` significa que x se repete zero ou mais vezes, `[ x ]` que x é opcional e `|` separa alternativas. Os textos entre aspas são tokens, e `IDENT`, `NUMERO` e `TEXTO` são as categorias de token da seção 4.

Código 5 – Gramática da FogeScript em EBNF

```
programa    = { funcao | declaracao } , inicio , { funcao | declaracao } ;
inicio      = "A SENHORA TRABALHA NA ASSEMBLEIA?" , bloco , "FUGIU" ;
funcao      = "NÃO, EU NUNCA FIZ ISSO" , IDENT ,
              "(" , [ IDENT , { "," , IDENT } ] , ")" , bloco , "FUGIU" ;
bloco       = { comando } ;

comando     = declaracao | saida | entrada | se | enquanto | para | parar | pular
            | retorno | tentativa | lancamento | expressao , [ ";" ] ;
declaracao  = "A SENHORA FEZ" , item , { "," , item } , [ ";" ] ;
item        = IDENT , [ "=" , expressao ] ;
saida       = "TEMOS GRAVADO!" , "(" , [ argumentos ] , ")" , [ ";" ] ;
entrada     = "QUER CONVERSAR COMIGO?" , "(" , IDENT , { "," , IDENT } , ")" , [ ";" ] ;
se          = "SE A SENHORA NÃO É FUNCIONÁRIA" , condicao , bloco ,
              { "ENTÃO A SENHORA É O QUÊ?" , condicao , bloco } ,
              [ "E TÁ CORRENDO POR QUÊ?" , bloco ] , "FUGIU" ;
condicao    = "(" , expressao , ")" ;
enquanto    = "ENQUANTO TIVER CORRENDO" , condicao , bloco , "FUGIU" ;
para        = "SENHORA? SENHORA?!" , "(" , [ inicio_para ] , ";" , [ expressao ] , ";" ,
              [ expressao ] , ")" , bloco , "FUGIU" ;
inicio_para = "A SENHORA FEZ" , item , { "," , item } | expressao ;
parar       = "VOLTA AQUI!" , [ ";" ] ;
pular       = "DEIXA ELA CORRER" , [ ";" ] ;
retorno     = "CHEGA DE CORRER" , [ expressao ] , [ ";" ] ;
tentativa   = "DEIXA EU TE PERGUNTAR" , bloco ,
              "A SENHORA TÁ NERVOSA?" , [ "(" , IDENT , ")" ] , bloco , "FUGIU" ;
lancamento  = "ME DEIXA EM PAZ!" , expressao , [ ";" ] ;

expressao   = ou , [ atribuidor , expressao ] ;
atribuidor  = "=" | "+=" | "-=" | "*=" | "/=" | "%=" ;
ou          = e , { "||" , e } ;
e           = igualdade , { "&&" , igualdade } ;
igualdade   = comparacao , { ( "==" | "!=" ) , comparacao } ;
comparacao  = soma , { ( "<" | ">" | "<=" | ">=" ) , soma } ;
soma        = produto , { ( "+" | "-" ) , produto } ;
produto     = unario , { ( "*" | "/" | "%" ) , unario } ;
unario      = ( "!" | "-" | "++" | "--" ) , unario | posfixo ;
posfixo     = primario , { "[" , expressao , "]" | "++" | "--" } ;
primario    = NUMERO | TEXTO | "TÁ GRAVADO" | "NÃO TÁ GRAVADO" | chamada | IDENT
            | "(" , expressao , ")" | "[" , [ argumentos ] , "]" ;
chamada     = [ "CHAMA A SENHORA" ] , IDENT , "(" , [ argumentos ] , ")" ;
argumentos  = expressao , { "," , expressao } ;
```

A precedência dos operadores sai da própria gramática, porque cada nível chama o nível de precedência maior. Da menor para a maior, a ordem fica: atribuição (associativa à direita), `||`, `&&`, `==` e `!=`, comparações, `+` e `-`, `*`, `/` e `%`, operadores unários e, por último, a posição de lista e o `++`/`--` depois do nome. Na regra `expressao`, o lado esquerdo de uma atribuição precisa ser uma variável ou uma posição de lista. Isso é conferido pelo analisador depois de ler a expressão.

5.2 Árvore sintática

Para o programa `somador.foge` do Código 1, o tradutor monta a árvore do Código 6. O tradutor mostra essa árvore quando é chamado com a opção `--arvore`. O tipo entre ⟨ ⟩ é anotado depois, pela análise semântica.

Código 6 – Árvore sintática do somador.foge

```
Programa
├── Função SOMAR(A, B)  · linha 2
│   └── Retorno (CHEGA DE CORRER)  · linha 3
│       └── Conta +
│           ├── Variável A
│           └── Variável B
└── Assembleia (A SENHORA TRABALHA NA ASSEMBLEIA?)  · linha 6
    ├── Declaração (A SENHORA FEZ)  · linha 7
    │   ├── variável A
    │   ├── variável B
    │   └── variável RES
    ├── Saída (TEMOS GRAVADO!)  · linha 8
    │   └── Texto "Me passa dois números aí, senhora:"
    ├── Entrada (QUER CONVERSAR COMIGO?) A, B  · linha 9
    ├── Atribuição =  · linha 10
    │   ├── Variável RES
    │   └── Chamada SOMAR(...)
    │       ├── Variável A
    │       └── Variável B
    ├── Saída (TEMOS GRAVADO!)  · linha 11
    │   ├── Texto "O resultado é"
    │   └── Variável RES
    └── Se (SE A SENHORA NÃO É FUNCIONÁRIA)  · linha 13
        ├── condição
        │   └── Comparação ==  ⟨booleano⟩
        │       ├── Variável RES
        │       └── Número 37
        ├── então
        │   └── Saída (TEMOS GRAVADO!)  · linha 14
        │       └── Texto "37! Tá gravado!"
        └── senão (E TÁ CORRENDO POR QUÊ?)
            └── Saída (TEMOS GRAVADO!)  · linha 16
                └── Texto "Volta aqui, senhora!"
```

5.3 Erros sintáticos

O analisador sintático para no primeiro erro e diz o que esperava encontrar. Entre os erros detectados estão bloco sem o `FUGIU` de fechamento, `FUGIU` sobrando, parêntese, colchete ou ponto e vírgula faltando, comando incompleto (como `A SENHORA FEZ X = ;`), senão sem o se, try sem catch, palavra reservada usada como nome, função declarada dentro de outro bloco e código fora da Assembleia.

Os três testes de erro sintático cobrem os casos pedidos na atividade: estrutura malformada (Código 7), falta de delimitador (Código 8) e comando incompleto (Código 9).

Código 7 – Erro sintático: bloco sem o FUGIU de fechamento

```
Erro sintático na linha 11, coluna 10: Faltou o "FUGIU" para fechar o "A SENHORA TRABALHA
NA ASSEMBLEIA?" da linha 5
  11 |     FUGIU
     |          ^
```

Código 8 – Erro sintático: parêntese sem fechar

```
Erro sintático na linha 6, coluna 45: Esperava ")" para fechar a condição, mas veio
"TEMOS GRAVADO!"
  6 |     SE A SENHORA NÃO É FUNCIONÁRIA (20 >= 18
    |                                             ^
```

Código 9 – Erro sintático: comando incompleto

```
Erro sintático na linha 5, coluna 29: Esperava um valor, mas veio ";"
  5 |     A SENHORA FEZ salario = ;
    |                             ^
```

6 ANÁLISE SEMÂNTICA

Depois que a árvore é montada, a função `analisar` percorre a AST e confere se o programa faz sentido: se os nomes usados existem, se as funções são chamadas do jeito certo e se os tipos das contas combinam. Quando há erro, o código não é gerado e todos os erros encontrados são mostrados de uma vez, com linha e coluna.

6.1 Escopos e tabela de símbolos

Durante a análise é montada uma tabela de símbolos com cada variável, parâmetro e função do programa. As regras de escopo são parecidas com as do JavaScript:

- as variáveis declaradas fora da Assembleia e das funções são globais;
- cada função enxerga os seus parâmetros, as suas variáveis e as globais, mas não enxerga as variáveis da Assembleia;
- cada bloco (se, senão, laços, try e catch) abre um escopo novo, e uma variável declarada dentro dele deixa de existir no `FUGIU`;
- uma variável precisa ser declarada antes de ser usada, mas uma função pode ser chamada antes da declaração, o que permite recursão.

A Tabela 6 mostra a tabela de símbolos do teste `03-folha-de-ponto.foge` (seção 8.4). Ela é mostrada pelo tradutor com a opção `--simbolos`.

Tabela 6 – Tabela de símbolos do teste 03-folha-de-ponto.foge

Nome	Espécie	Tipo deduzido	Escopo	Linha:coluna
`lista`	parâmetro	desconhecido	função somar	6:30
`total`	variável	desconhecido	função somar	7:19
`i`	variável	número	laço da linha 8	8:39
`media`	parâmetro	desconhecido	função classificar	14:36
`nome`	variável	desconhecido	Assembleia	24:19
`dias`	variável	desconhecido	Assembleia	24:25
`horas`	variável	lista	Assembleia	25:19
`d`	variável	número	laço da linha 28	28:39
`h`	variável	desconhecido	bloco da linha 28	29:23
`total`	variável	desconhecido	Assembleia	34:19
`media`	variável	número	Assembleia	35:19

6.2 Inferência de tipos

Como a declaração não diz o tipo, o analisador descobre o tipo de cada variável olhando todos os valores que ela recebe no programa: o valor inicial, as atribuições, o `++` e o `--` e as leituras da entrada. Um tipo pode depender de outro (em `X = Y + 1`, o tipo de `X` depende do tipo de `Y`), então esse cálculo é repetido até nenhum tipo mudar. Se todos os valores de uma variável têm o mesmo tipo, ela fica com esse tipo. Se ela recebe valores de tipos diferentes, vem da entrada ou é parâmetro de função, o tipo fica desconhecido e a conferência é feita quando o programa roda. É por isso que, na Tabela 6, `i` e `d` são número, `horas` é lista e `nome` é desconhecido.

6.3 Verificações

Tabela 7 – Verificações feitas pela análise semântica

Verificação	Exemplo de mensagem
Variável não declarada	A senhora nunca fez "totl". Declare antes com "A SENHORA FEZ totl". Você quis dizer "total"?
Variável declarada duas vezes no mesmo bloco	A senhora já fez "nome" neste bloco (na linha 5). Não dá para declarar de novo
Função que não existe	A senhora não conhece a função "SOMRA". [...] Você quis dizer "SOMAR"?
Chamada com o número errado de valores	A função "SOMAR" espera 2 valor(es), mas recebeu 1
Função usada como variável, ou o contrário	"DOBRO" é uma função, não uma variável. Para chamar, use DOBRO(...)
`VOLTA AQUI!` ou `DEIXA ELA CORRER` fora de um laço	"VOLTA AQUI!" só pode ser usado dentro de um laço
Conta com tipo inválido	A conta "*" só funciona com números, e recebeu texto e número
Comparação entre tipos diferentes	Não dá para comparar texto com número usando "<"
Variável sem valor usada numa conta	Não dá para somar nada com número. A variável "X" foi declarada sem valor e nunca recebeu um
`++`, `--` ou `-` em algo que não é número	"++" só funciona com número, e a variável tem texto
Divisão por zero escrita no código	Divisão por zero! Nem a Assembleia aprova isso

A análise também gera avisos, que não impedem a tradução: variável que recebe valor mas nunca é usada e comparação com `==` entre tipos diferentes, que sempre dá falso. Os códigos a seguir mostram três dos testes de erro semântico: variável não declarada, tipos incompatíveis e um comando usado onde não pode.

Código 10 – Erro semântico: variável não declarada

```
Erro semântico na linha 9, coluna 20: A senhora nunca fez "totl". Declare antes com "A
SENHORA FEZ totl". Você quis dizer "total"?
  9 |     TEMOS GRAVADO!(totl);
    |                    ^
```

Código 11 – Erro semântico: tipos incompatíveis

```
Erro semântico na linha 7, coluna 39: A conta "*" só funciona com números, e recebeu
texto e número
  7 |     TEMOS GRAVADO!("Em dobro:", idade * 2);
    |                                       ^
```

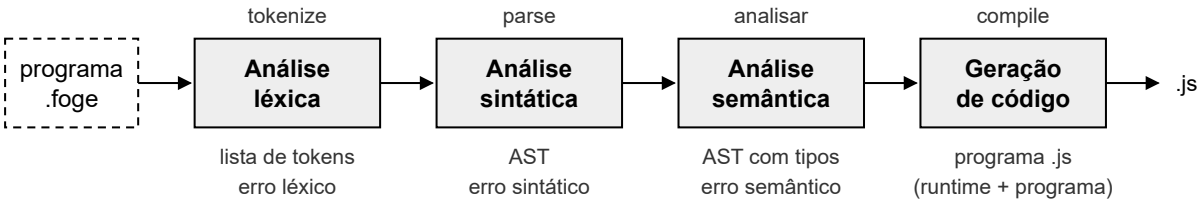
Código 12 – Erro semântico: VOLTA AQUI! fora de um laço

```
Erro semântico na linha 7, coluna 9: "VOLTA AQUI!" só pode ser usado dentro de um laço
  7 |         VOLTA AQUI!;
    |         ^
```

7 GERAÇÃO DE CÓDIGO

A última etapa é a função `compile`, que percorre a AST já conferida e escreve o JavaScript equivalente. A estrutura do programa é mantida: cada comando FogeScript vira o comando correspondente da Tabela 1, os blocos viram chaves e a Assembleia vira uma função `assembleia()`, chamada no fim do arquivo. A Figura 1 resume o caminho de um programa pelo tradutor.

Figura 1 – Etapas do tradutor FogeScript



Acima de cada etapa está a função do `fogescript.js` que a implementa. Embaixo, o que ela produz e o tipo de erro que ela pode acusar.

O gerador usa os tipos da análise semântica para decidir como escrever cada conta. Quando os dois lados com certeza são números (ou textos), a conta vira o operador do próprio JavaScript, como `N % 15 === 0`. Quando o tipo só é conhecido na hora de rodar, por exemplo um valor lido da entrada, a conta vira uma chamada ao runtime, como `foge.conta(">", palpite, DIAS_TRABALHADOS, 21)`. Essa função confere os tipos e, se der erro, avisa a linha do programa FogeScript, que é o último número da chamada.

O runtime é um conjunto pequeno de funções de apoio, parecido com uma biblioteca padrão: `foge.gravar` (saída), `foge.ler` (entrada), `foge.conta` (contas), `foge.item` e `foge.guardar` (listas), `foge.chamar` (chamadas de função), `foge.lancar` e `foge.capturar` (erros) e `foge.passo`, que interrompe um laço infinito depois de 5 milhões de voltas. O arquivo `.js` gerado traz o runtime junto com o programa traduzido, então ele roda sozinho com `node programa.js`, sem precisar do FogeScript.

O site também tem um interpretador, que executa a AST direto, sem gerar JavaScript. Ele usa as mesmas funções do runtime, e os testes conferem se os dois modos dão a mesma saída.

O código traduzido do teste `02-adivinhacao.foge`, na seção 8.3, mostra os dois casos. A comparação `tentativas < MAX_TENTATIVAS` virou o operador do JavaScript, porque as duas variáveis são números, enquanto `palpite > DIAS_TRABALHADOS` virou `foge.conta`, porque `palpite` vem da entrada e pode ser texto.

8 TESTES E EVIDÊNCIAS DE FUNCIONAMENTO

8.1 Programas de teste

Os programas de teste ficam na pasta `testes`. São 5 programas válidos, em `testes/validos`, e 12 programas com erro, em `testes/erros`. Cada programa válido tem um arquivo `.saida` com a saída esperada, que escrevemos à mão, e um arquivo `.entrada` quando o programa lê dados. Cada programa com erro diz na primeira linha qual erro deve aparecer, por exemplo `// ESPERADO: erro sintático na linha 6, coluna 45`.

Tabela 8 – Programas de teste válidos (pasta testes/validos)

Arquivo	O que testa
`01-tipos-e-operadores.foge`	Tipos de dados, as quatro operações, resto, comparações, lógica e comentários
`02-adivinhacao.foge`	Entrada de dados, decisões (se, senão se, senão) e repetição (enquanto)
`03-folha-de-ponto.foge`	Funções, listas, laço para e entrada de dados
`04-recursao-e-escopo.foge`	Recursão, variável global e escopo de bloco
`05-tratamento-de-erros.foge`	Try/catch e throw com erros na hora de rodar

Tabela 9 – Programas de teste com erro (pasta testes/erros)

Arquivo	O que tem de errado	Erro esperado (etapa, linha:coluna)
`lexico-1-simbolo-estranho.foge`	Símbolo `@`, que não é da linguagem	léxico, 6:30
`lexico-2-texto-sem-fechar.foge`	Texto sem as aspas de fechamento	léxico, 5:20
`lexico-3-nome-com-numero.foge`	Nome começando com número	léxico, 5:19
`sintatico-1-faltou-fugiu.foge`	Bloco sem o `FUGIU`	sintático, 11:10
`sintatico-2-faltou-parentese.foge`	Condição sem o `)`	sintático, 6:45
`sintatico-3-comando-incompleto.foge`	Atribuição sem valor	sintático, 5:29
`semantico-1-variavel-nao-declarada.foge`	Variável não declarada	semântico, 9:20
`semantico-2-tipos-incompativeis.foge`	Texto vezes número	semântico, 7:39
`semantico-3-funcao-errada.foge`	Função com um valor a menos	semântico, 10:20
`semantico-4-declarada-duas-vezes.foge`	Variável declarada duas vezes	semântico, 6:19
`semantico-5-volta-aqui-fora-do-laco.foge`	`VOLTA AQUI!` fora de laço	semântico, 7:9
`execucao-1-divisao-por-zero.foge`	Divisão por zero vinda da entrada	execução, linha 10

8.2 Execução dos testes

O script `testes/rodar-testes.js` roda todos os testes de uma vez. Cada programa válido é executado de três formas: pelo interpretador, pelo JavaScript gerado rodando dentro do próprio tradutor e pelo arquivo `.js` gerado rodando no Node. As três saídas precisam ser iguais à saída esperada. Nos programas com erro, o script confere se o erro saiu na etapa, na linha e na coluna esperadas. Ele também roda os 7 exemplos do site e confere se os três modos dão a mesma saída.

As evidências de cada teste (código original, entrada, código traduzido e resultado) ficam gravadas na pasta `testes/resultados`. O Código 13 é o resultado da última execução do script.

Código 13 – Saída de node testes/rodar-testes.js (arquivo testes/resultados/resumo.txt)

```
FogeScript 1.2.0: resultado dos testes

Programas válidos:  5/5
Programas com erro: 12/12
Exemplos do site:   7/7

ok     validos   01-tipos-e-operadores  (saída igual à esperada)
ok     validos   02-adivinhacao  (saída igual à esperada)
ok     validos   03-folha-de-ponto  (saída igual à esperada)
ok     validos   04-recursao-e-escopo  (saída igual à esperada)
ok     validos   05-tratamento-de-erros  (saída igual à esperada)
ok     erros     execucao-1-divisao-por-zero  (de execução, linha 10)
ok     erros     lexico-1-simbolo-estranho  (léxico, linha 6, coluna 30)
ok     erros     lexico-2-texto-sem-fechar  (léxico, linha 5, coluna 20)
ok     erros     lexico-3-nome-com-numero  (léxico, linha 5, coluna 19)
ok     erros     semantico-1-variavel-nao-declarada  (semântico, linha 9, coluna 20)
ok     erros     semantico-2-tipos-incompativeis  (semântico, linha 7, coluna 39)
ok     erros     semantico-3-funcao-errada  (semântico, linha 10, coluna 20)
ok     erros     semantico-4-declarada-duas-vezes  (semântico, linha 6, coluna 19)
ok     erros     semantico-5-volta-aqui-fora-do-laco  (semântico, linha 7, coluna 9)
ok     erros     sintatico-1-faltou-fugiu  (sintático, linha 11, coluna 10)
ok     erros     sintatico-2-faltou-parentese  (sintático, linha 6, coluna 45)
ok     erros     sintatico-3-comando-incompleto  (sintático, linha 5, coluna 29)
ok     exemplos  erros  (os três jeitos de rodar deram a mesma saída)
ok     exemplos  fatorial  (os três jeitos de rodar deram a mesma saída)
ok     exemplos  fizzbuzz  (os três jeitos de rodar deram a mesma saída)
ok     exemplos  flagrante  (os três jeitos de rodar deram a mesma saída)
ok     exemplos  listas  (os três jeitos de rodar deram a mesma saída)
ok     exemplos  ola  (os três jeitos de rodar deram a mesma saída)
ok     exemplos  somador  (os três jeitos de rodar deram a mesma saída)

Todos os testes passaram.
```

8.3 Evidência 1: entrada de dados, decisões e repetição

O teste `02-adivinhacao.foge` é o programa com entrada de dados, decisões e repetição pedido na atividade. O programa tenta adivinhar quantos dias a senhora trabalhou no mês: cada linha da entrada é um palpite, e o laço para quando o palpite está certo ou quando acabam as tentativas.

Código 14 – Código original: testes/validos/02-adivinhacao.foge

```
/*
    Teste 2: entrada de dados, decisões e repetição.
    A repórter tenta adivinhar quantos dias a senhora trabalhou no mês.
    Cada linha da Entrada é um palpite. O laço para quando ela acerta
    ou quando acabam as tentativas.
*/
A SENHORA FEZ DIAS_TRABALHADOS = 3;    // o segredo
A SENHORA FEZ MAX_TENTATIVAS = 5;

A SENHORA TRABALHA NA ASSEMBLEIA?
    A SENHORA FEZ palpite;
    A SENHORA FEZ tentativas = 0;
    A SENHORA FEZ acertou = NÃO TÁ GRAVADO;

    TEMOS GRAVADO!("Repórter: Quantos dias a senhora trabalhou este mês?");
    ENQUANTO TIVER CORRENDO (!acertou && tentativas < MAX_TENTATIVAS)
        QUER CONVERSAR COMIGO?(palpite);
        tentativas++;
        SE A SENHORA NÃO É FUNCIONÁRIA (palpite == DIAS_TRABALHADOS)
            acertou = TÁ GRAVADO;
        ENTÃO A SENHORA É O QUÊ? (palpite > DIAS_TRABALHADOS)
            TEMOS GRAVADO!(palpite, "dias? Menos, senhora, menos.");
        E TÁ CORRENDO POR QUÊ?
            TEMOS GRAVADO!(palpite, "dias? Mais que isso!");
        FUGIU
    FUGIU

    SE A SENHORA NÃO É FUNCIONÁRIA (acertou)
        TEMOS GRAVADO!("TEMOS GRAVADO! Foram", DIAS_TRABALHADOS, "dias. Acertou em",
tentativas, "tentativa(s).");
    E TÁ CORRENDO POR QUÊ?
        TEMOS GRAVADO!("Ela fugiu antes de responder.");
    FUGIU
FUGIU
```

Código 15 – Código traduzido para JavaScript (sem o runtime)

```
'use strict';
// Programa FogeScript traduzido para JavaScript (tradutor 1.2.0).
// Contas com tipos garantidos pela análise semântica usam os operadores do JavaScript.
// As outras chamam o runtime (foge.*), que confere os tipos na hora de rodar.
// O último número de cada chamada foge.* é a linha do código FogeScript.

// Variáveis globais
let DIAS_TRABALHADOS = 3;
let MAX_TENTATIVAS = 5;

// A SENHORA TRABALHA NA ASSEMBLEIA?
function assembleia() {
  let palpite;
  let tentativas = 0;
  let acertou = false;
  foge.gravar("Repórter: Quantos dias a senhora trabalhou este mês?");
  while (!acertou && tentativas < MAX_TENTATIVAS) {
    foge.passo(16);
    palpite = foge.ler(17);
    tentativas++;
    if (palpite === DIAS_TRABALHADOS) {
      acertou = true;
    } else if (foge.conta(">", palpite, DIAS_TRABALHADOS, 21)) {
      foge.gravar(palpite, "dias? Menos, senhora, menos.");
    } else {
      foge.gravar(palpite, "dias? Mais que isso!");
    }
  }
  if (acertou) {
    foge.gravar("TEMOS GRAVADO! Foram", DIAS_TRABALHADOS, "dias. Acertou em", tentativas,
"tentativa(s).");
  } else {
    foge.gravar("Ela fugiu antes de responder.");
  }
}
assembleia();
```

Resultado da execução do arquivo gerado, com a entrada 10, 1, 5 e 3 (um palpite por linha):

Código 16 – Resultado da execução

```
$ node 02-adivinhacao.js < 02-adivinhacao.entrada
Repórter: Quantos dias a senhora trabalhou este mês?
10 dias? Menos, senhora, menos.
1 dias? Mais que isso!
5 dias? Menos, senhora, menos.
TEMOS GRAVADO! Foram 3 dias. Acertou em 4 tentativa(s).
```

8.4 Evidência 2: funções, listas e laço para

O teste `03-folha-de-ponto.foge` lê o nome de uma funcionária, quantos dias ela trabalhou e as horas de cada dia. Depois calcula o total e a média com uma função e classifica a funcionária com outra.

Código 17 – Código original: testes/validos/03-folha-de-ponto.foge

```
/*
    Teste 3: funções, listas, laço SENHORA? SENHORA?! (for) e decisões encadeadas.
    Entrada: o nome da funcionária, quantos dias ela trabalhou e,
    depois, as horas de cada dia (uma resposta por linha).
*/
NÃO, EU NUNCA FIZ ISSO somar(lista)
    A SENHORA FEZ total = 0;
    SENHORA? SENHORA?! (A SENHORA FEZ i = 0; i < TAMANHO(lista); i++)
        total += lista[i];
    FUGIU
    CHEGA DE CORRER total;
FUGIU

NÃO, EU NUNCA FIZ ISSO classificar(media)
    SE A SENHORA NÃO É FUNCIONÁRIA (media >= 8)
        CHEGA DE CORRER "funcionária exemplar";
    ENTÃO A SENHORA É O QUÊ? (media >= 4)
        CHEGA DE CORRER "meio expediente";
    FUGIU
    CHEGA DE CORRER "funcionária fantasma";
FUGIU

A SENHORA TRABALHA NA ASSEMBLEIA?
    A SENHORA FEZ nome, dias;
    A SENHORA FEZ horas = [];

    QUER CONVERSAR COMIGO?(nome, dias);
    SENHORA? SENHORA?! (A SENHORA FEZ d = 1; d <= dias; d++)
        A SENHORA FEZ h;
        QUER CONVERSAR COMIGO?(h);
        horas[TAMANHO(horas)] = h;
    FUGIU

    A SENHORA FEZ total = somar(horas);
    A SENHORA FEZ media = total / TAMANHO(horas);
    TEMOS GRAVADO!("Funcionária:", nome);
    TEMOS GRAVADO!("Horas por dia:", horas);
    TEMOS GRAVADO!("Total:", total, "horas | Média:", media);
    TEMOS GRAVADO!("Veredito:", classificar(media));
FUGIU
```

Código 18 – Código traduzido para JavaScript (sem o runtime)

```
'use strict';
// Programa FogeScript traduzido para JavaScript (tradutor 1.2.0).
// Contas com tipos garantidos pela análise semântica usam os operadores do JavaScript.
// As outras chamam o runtime (foge.*), que confere os tipos na hora de rodar.
// O último número de cada chamada foge.* é a linha do código FogeScript.

// Funções (NÃO, EU NUNCA FIZ ISSO)
function somar(lista) {
  let total = 0;
  for (let i = 0; i < foge.chamar("TAMANHO", [lista], 8); i++) {
    foge.passo(8);
    total = foge.acumular("+=", foge.item(lista, i, 9), total, 9);
  }
  return total;
}
foge.funcao("somar", somar);

function classificar(media) {
  if (foge.conta(">=", media, 8, 15)) {
    return "funcionária exemplar";
  } else if (foge.conta(">=", media, 4, 17)) {
    return "meio expediente";
  }
  return "funcionária fantasma";
}
foge.funcao("classificar", classificar);

// A SENHORA TRABALHA NA ASSEMBLEIA?
function assembleia() {
  let nome;
  let dias;
  let horas = [];
  nome = foge.ler(27);
  dias = foge.ler(27);
  for (let d = 1; foge.conta("<=", d, dias, 28); d++) {
    foge.passo(28);
    let h;
    h = foge.ler(30);
    foge.guardar(h, foge.lista(horas, 31), foge.chamar("TAMANHO", [horas], 31), 31);
  }
  let total = foge.chamar("somar", [horas], 34);
  let media = foge.conta("/", total, foge.chamar("TAMANHO", [horas], 35), 35);
  foge.gravar("Funcionária:", nome);
  foge.gravar("Horas por dia:", horas);
  foge.gravar("Total:", total, "horas | Média:", media);
  foge.gravar("Veredito:", foge.chamar("classificar", [media], 39));
}
assembleia();
```

Resultado da execução com a entrada Dona Cida, 4, 6, 8, 3 e 5:

Código 19 – Resultado da execução

```
$ node 03-folha-de-ponto.js < 03-folha-de-ponto.entrada
Funcionária: Dona Cida
Horas por dia: [6, 8, 3, 5]
Total: 22 horas | Média: 5.5
Veredito: meio expediente
```

8.5 Evidência 3: erro na hora de rodar

O teste `execucao-1-divisao-por-zero.foge` não tem erro de escrita, então passa pelas três análises e é traduzido. O erro só aparece quando ele roda com a entrada 10 e 0, porque a divisão por zero depende do valor digitado. O programa para e mostra a linha do código FogeScript onde o erro aconteceu.

Código 20 – Tradução e execução do teste execucao-1-divisao-por-zero.foge

```
$ node tradutor.js testes/erros/execucao-1-divisao-por-zero.foge --rodar <
testes/erros/execucao-1-divisao-por-zero.entrada
FogeScript 1.2.0 · tradutor para JavaScript
Arquivo: testes/erros/execucao-1-divisao-por-zero.foge

  1. Análise léxica..... ok   34 tokens
  2. Análise sintática... ok   árvore com 13 nós
  3. Análise semântica... ok   2 nomes, 0 funções, 0 avisos
  4. Geração de código... ok   testes/erros/execucao-1-divisao-por-zero.js

Rodando: node testes/erros/execucao-1-divisao-por-zero.js

Dividindo 10 por 0

Erro de execução na linha 10: Divisão por zero! Nem a Assembleia aprova isso
```

Os outros três testes válidos (`01`, `04` e `05`) e os testes de erro que não aparecem neste documento também têm as evidências gravadas na pasta `testes/resultados`.

9 COMO COMPILAR E EXECUTAR

O tradutor não precisa ser compilado, porque é escrito em JavaScript. Para usar pela linha de comando é preciso ter o Node.js instalado (versão 18 ou mais nova). Depois de baixar o projeto do GitHub, os comandos da Tabela 10 são executados dentro da pasta do projeto.

Tabela 10 – Comandos do tradutor

Comando	O que faz
`node tradutor.js programa.foge`	Traduz o programa e grava `programa.js`
`node programa.js`	Roda o programa traduzido, lendo a entrada do teclado
`node programa.js < entrada.txt`	Roda o programa lendo a entrada de um arquivo
`node tradutor.js programa.foge --rodar`	Traduz e já roda o programa
`node tradutor.js programa.foge --tokens`	Mostra os tokens (análise léxica)
`node tradutor.js programa.foge --arvore`	Mostra a árvore sintática
`node tradutor.js programa.foge --simbolos`	Mostra a tabela de símbolos
`node tradutor.js programa.foge -o saida.js`	Escolhe o nome do arquivo gerado
`node testes/rodar-testes.js`	Roda todos os testes e grava as evidências

No PowerShell do Windows o símbolo < não funciona. Nesse caso, a entrada pode ser passada com `Get-Content entrada.txt | node programa.js`.

O Código 21 mostra a saída do tradutor para o `somador.foge`. Primeiro aparece o resultado de cada etapa e, no fim, o nome do arquivo gerado.

Código 21 – Saída do tradutor para um programa sem erros

```
$ node tradutor.js exemplos/somador.foge
FogeScript 1.2.0 · tradutor para JavaScript
Arquivo: exemplos/somador.foge

  1. Análise léxica...... ok   69 tokens
  2. Análise sintática... ok   árvore com 27 nós
  3. Análise semântica... ok   5 nomes, 1 função, 0 avisos
  4. Geração de código... ok   exemplos/somador.js

Para rodar: node exemplos/somador.js
```

Para usar sem instalar nada, basta abrir o site https://fogescript.vercel.app. Ele tem um editor com os exemplos e os programas de teste, e o botão Compilar mostra as quatro etapas da tradução em abas (tokens, árvore, análise semântica e JavaScript gerado), além de permitir baixar o arquivo `.js`. A Tabela 11 mostra a organização do projeto.

Tabela 11 – Arquivos do projeto

Arquivo ou pasta	Conteúdo
`fogescript.js`	O tradutor: análise léxica (`tokenize`), sintática (`parse`), semântica (`analisar`), geração de código (`compile`), runtime e interpretador
`tradutor.js`	Programa de linha de comando que usa o `fogescript.js`
`index.html`	Site com o editor, as etapas da tradução em abas, o dicionário e o tutorial
`exemplos/`	Programas de exemplo usados no site
`testes/`	Programas válidos e com erro, o script de testes e as evidências (`testes/resultados`)
`docs/`	Este documento

10 CONSIDERAÇÕES FINAIS

Com o trabalho, conseguimos aplicar na prática as etapas de um compilador vistas na disciplina. O tradutor não faz uma simples troca de palavras: o código passa pela análise léxica, pela análise sintática, que monta a árvore, e pela análise semântica, que confere nomes e tipos, e só então o JavaScript é gerado a partir da árvore.

A principal particularidade da FogeScript ficou na análise léxica. Como as palavras reservadas são frases com espaços, pontuação e acentos, não dava para separar os tokens só pelos espaços, e foi preciso usar a regra do maior casamento. Na análise semântica, a falta de tipo na declaração fez com que o tipo de cada variável tivesse que ser deduzido pelos valores que ela recebe.

Como melhorias futuras, a linguagem poderia ter declaração de tipo opcional, mais funções prontas e geração de código para outras linguagens, como C ou Python, aproveitando a mesma árvore sintática.

REFERÊNCIAS

BIRL: BAMBAM'S "IT'S SHOW TIME" RECURSIVE LANGUAGE. **Repositório da linguagem BIRL**. Disponível em: https://github.com/birl-language/birl. Acesso em: 4 nov. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 14977**: Information technology: Syntactic metalanguage: Extended BNF. Genebra: ISO, 1996.

NODE.JS. **Node.js documentation**. Disponível em: https://nodejs.org/docs. Acesso em: 4 nov. 2026.